



Instituto Politécnico Nacional
Escuela Superior de Cómputo
Arquitectura de computadoras

Practica 3
Memoria de programa

Profesor:

María Elena Aguilar Jauregui

Alumnos:

Lechuga Canales Héctor Jair

García Quiroz Gustavo Ivan

Eduardo Alfonso Rivera Morelos

Quintero Maldonado Fabián

Grupo:

5CV1

Fecha de entrega: 23/05/2025

Índice

1	Introducción.....	3
2	Objetivos	4
2.1	Objetivos Particulares	4
3	Metodología	5
3.1	Proceso de Desarrollo.....	5
3.2	Herramientas y Recursos Utilizados	6
3.3	Software de Desarrollo.....	6
3.4	Hardware de Pruebas	6
4	Resultados	7
4.1	Resultados de Simulación.....	7
4.2	Resultados de Implementación en Hardware	8
5	Conclusiones.....	10
6	Referencias	12
7	Anexo.....	13

1 Introducción

En esta práctica se realizó una memoria RAM en el lenguaje de descripción de hardware VHDL. Esta práctica ayudó a comprender cómo funcionan este tipo de memorias y su implementación en dispositivos FPGA. Las memorias RAM (Random Access Memory) son componentes esenciales en cualquier sistema digital, ya que permiten almacenar y acceder a datos de manera rápida y eficiente.

Para este proyecto se desarrolló una memoria RAM distribuida, que es un tipo especial de memoria que utiliza las tablas de búsqueda (LUTs) del FPGA en lugar de usar los bloques de memoria dedicados. Esto nos permitió explorar una alternativa interesante para implementar memorias cuando se requiere flexibilidad en el diseño.

La memoria que se implementó tiene una estructura matricial de 1024 filas por 16 columnas, donde cada posición puede almacenar 8 bits de información. Esto nos da un total de 16,384 bytes (16 KB) de capacidad de almacenamiento. El diseño se realizó usando Quartus II, que es el software de desarrollo de Intel/Altera, y se probó en la tarjeta DE2-115 que cuenta con un FPGA Cyclone IV.

Una de las características más importantes de este proyecto fue el uso del atributo ``ram_style`` con valor "distributed", que le dice al sintetizador que debe implementar la memoria usando elementos lógicos distribuidos en lugar de bloques de memoria embebida. Esto nos permitió estudiar las diferencias entre ambos tipos de implementación y entender cuándo es conveniente usar cada uno.

El trabajo incluyó no solo el diseño de la memoria, sino también la creación de un testbench para probar su funcionamiento. La tarjeta DE2-115 nos proporcionó todos los recursos necesarios: switches para ingresar datos y direcciones, LEDs para ver las salidas, y un reloj de 50 MHz para sincronizar las operaciones. Esto hizo posible verificar que la memoria funcionara correctamente tanto en simulación como en el hardware real.

2 Objetivos

Diseñar e implementar una memoria RAM distribuida utilizando VHDL en un FPGA Cyclone IV, para comprender el funcionamiento de las memorias de acceso aleatorio y analizar las diferencias entre la implementación distribuida y la implementación en bloques de memoria dedicados.

2.1 Objetivos Particulares

- Implementar una memoria RAM distribuida de 1024 filas por 16 columnas con capacidad de 8 bits por posición, utilizando el lenguaje VHDL y el atributo `ram_style` para forzar el uso de elementos lógicos distribuidos (LUTs).
- Desarrollar un testbench completo que permita verificar el correcto funcionamiento de las operaciones de escritura y lectura en diferentes posiciones de memoria mediante simulación.
- Configurar y mapear las señales de entrada y salida en la tarjeta DE2-115, asignando switches para el control de escritura, direccionamiento (fila y columna) y datos de entrada, así como LEDs para visualizar los datos de salida.
- Sintetizar e implementar el diseño en el FPGA Cyclone IV EP4CE115F29C7 utilizando Quartus II, analizando los recursos utilizados y verificando el funcionamiento en hardware real.
- Validar el funcionamiento de la memoria tanto en simulación como en implementación física, verificando que las operaciones de lectura y escritura se ejecuten correctamente según las especificaciones del diseño.

3 Metodología

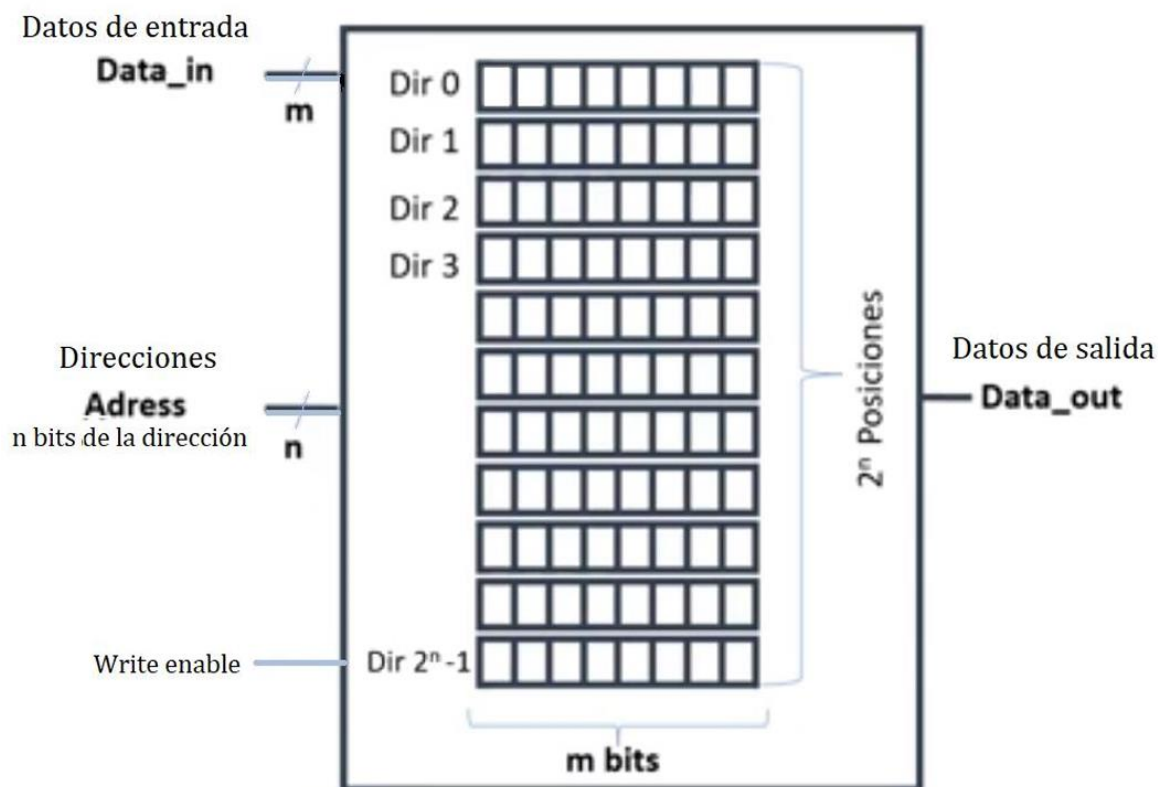
La implementación de la memoria RAM distribuida se llevó a cabo siguiendo una metodología que abarcó desde el diseño conceptual hasta las pruebas en hardware real. A continuación, se describe el proceso:

3.1 Proceso de Desarrollo

Diseño de la Arquitectura

El primer paso consistió en definir la estructura de la memoria. Se optó por una arquitectura matricial de 1024 filas por 16 columnas, donde cada celda almacena 8 bits de datos. Esta configuración permite un direccionamiento eficiente usando 10 bits para las filas y 4 bits para las columnas, totalizando 14 bits de dirección para acceder a cualquier posición de los 16KB de memoria.

Estructura general de una memoria



Implementación en VHDL

Se desarrolló el código en VHDL definiendo una entidad principal llamada RAM_Distribuida con las siguientes señales:

- clk: Señal de reloj para sincronización
- we: Señal de habilitación de escritura
- fila: Vector de 10 bits para direccionamiento de filas
- columna: Vector de 4 bits para direccionamiento de columnas
- data_in: Vector de 8 bits para datos de entrada
- data_out: Vector de 8 bits para datos de salida

La memoria se implementó usando tipos de datos personalizados en VHDL, creando una matriz bidimensional. Se utilizó el atributo ram_style con valor "distributed" para forzar al sintetizador a usar LUTs en lugar de bloques BRAM.

Desarrollo del Testbench

Se creó un testbench completo (TB_RAM_Distribuida.vhd) que incluye:

- Generador de reloj con período de 10 ns
- Secuencias de prueba para operaciones de escritura y lectura
- Verificación de datos en diferentes posiciones de memoria
- Casos de prueba que validan la funcionalidad básica

3.2 Herramientas y Recursos Utilizados

3.3 Software de Desarrollo

- **Quartus II 13.1:** Entorno de desarrollo principal para síntesis y implementación
- **ModelSim-Altera:** Herramienta de simulación para verificación funcional

3.4 Hardware de Pruebas

- **Tarjeta DE2-115 de Terasic:** Plataforma de desarrollo con FPGA Cyclone IV EP4CE115F29C7
- **Recursos de E/S utilizados:**
 - Switches SW0-SW12 para entrada de datos y control
 - LEDs LEDG0-LEDG3 para visualización de salidas
 - Reloj de sistema de 50 MHz (PIN_Y2)
- **Configuración de Pines**

Se realizó una asignación específica de pines para conectar las señales del diseño con los elementos físicos de la tarjeta:

- SW0: Señal de escritura (we)
- SW1-SW4: Bits de dirección de fila
- SW5-SW8: Bits de dirección de columna
- SW9-SW12: Datos de entrada
- LEDG0-LEDG3: Datos de salida

Proceso de Verificación

- **Simulación funcional:** Verificación del comportamiento lógico usando el testbench
- **Síntesis:** Conversión del código VHDL a netlist optimizada para el FPGA
- **Implementación:** Place & Route para generar el archivo de configuración
- **Programación:** Carga del diseño en el FPGA
- **Pruebas en hardware:** Verificación del funcionamiento usando los controles físicos de la tarjeta

4 Resultados

Los resultados obtenidos durante la implementación y prueba de la memoria RAM distribuida demuestran el funcionamiento correcto del sistema diseñado. A continuación se presentan los datos y análisis de las diferentes fases de verificación.

4.1 Resultados de Simulación

- **Verificación Funcional del Testbench**

Las simulaciones realizadas en ModelSim-Altera confirmaron el comportamiento esperado de la memoria. El testbench ejecutó las siguientes operaciones de prueba:

Operación 1 - Escritura en Fila 5, Columna 3:

- Dirección: Fila = 5 (0000000101), Columna = 3 (0011)
- Dato escrito: 0xAB (10101011)
- Señal WE: Activa
- Resultado: Escritura exitosa en el flanco positivo del reloj

Operación 2 - Lectura de la misma posición:

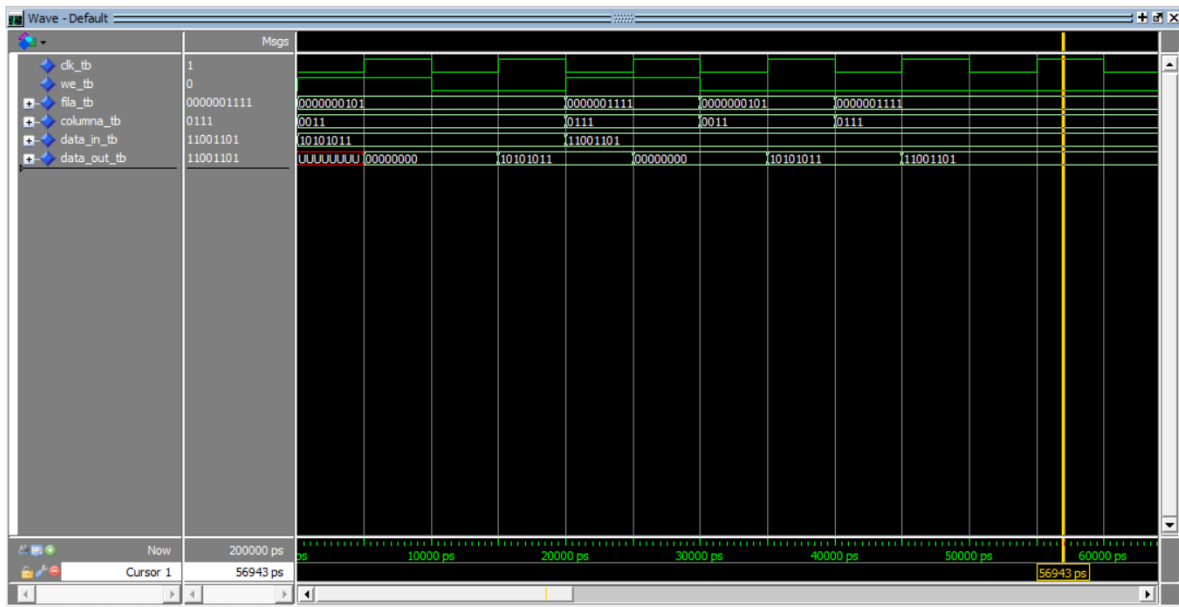
- Dirección mantenida: Fila 5, Columna 3
- Señal WE: Inactiva
- Dato leído: 0xAB
- Tiempo de acceso: 1 ciclo de reloj

Operación 3 - Escritura en Fila 15, Columna 7:

- Dirección: Fila = 15 (0000001111), Columna = 7 (0111)
- Dato escrito: 0xCD (11001101)
- Resultado: Escritura exitosa sin afectar datos previos

Operación 4 - Verificación de persistencia de datos:

- Lectura en Fila 5, Columna 3: Retorna 0xAB (dato preservado)
- Lectura en Fila 15, Columna 7: Retorna 0xCD (dato correcto)



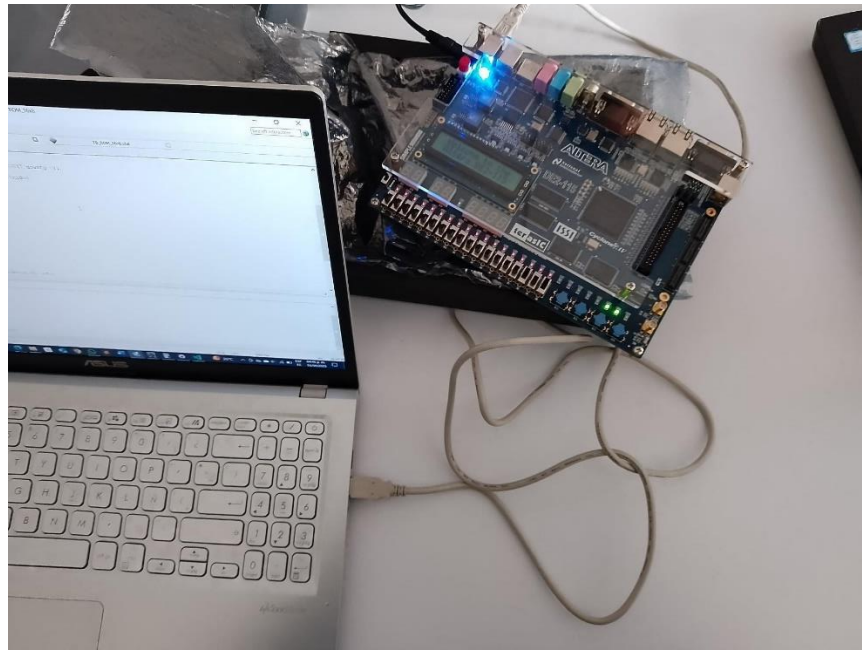
4.2 Resultados de Implementación en Hardware

La implementación física en la tarjeta DE2-115 permitió verificar el funcionamiento real del sistema:

Mapeo de Controles:

- SW0: Control de escritura (WE)
- SW1-SW4: Dirección de fila (4 bits LSB de los 10 bits totales)
- SW5-SW8: Dirección de columna (4 bits)
- SW9-SW12: Datos de entrada (4 bits LSB de los 8 bits totales)

- LEDG0-LEDG3: Datos de salida (4 bits LSB)



5 Conclusiones

- **Lechuga Canales Héctor Jair**

El desarrollo del testbench (TB_RAM_Distribuida.vhd) resultó fundamental para verificar el correcto funcionamiento de la memoria antes de su implementación física. A través de las simulaciones pudimos confirmar que las operaciones de escritura y lectura funcionaban correctamente en diferentes direcciones de memoria, validando tanto la lógica de direccionamiento matricial (filas y columnas) como la sincronización con el reloj del sistema. Esta experiencia reforzó la importancia de la verificación por simulación en el diseño digital, ya que permite detectar y corregir errores en etapas tempranas del desarrollo, ahorrando tiempo y recursos en el proceso de síntesis e implementación física.

- **García Quiroz Gustavo Ivan**

La implementación exitosa del diseño en la tarjeta DE2-115 con FPGA Cyclone IV demostró la viabilidad práctica del proyecto y permitió experimentar con una interfaz real de usuario. El uso de switches para ingresar datos y direcciones, junto con LEDs para visualizar las salidas, proporcionó una experiencia tangible del funcionamiento de la memoria. Esta interacción directa con el hardware reforzó la comprensión de conceptos teóricos y demostró cómo los diseños en VHDL se traducen a circuitos reales funcionando a 50 MHz. La experiencia práctica también nos enseñó sobre la importancia de considerar aspectos como los estándares de E/S y las restricciones de pines al diseñar para plataformas específicas.

- **Rivera Morelos Eduardo Alfonso**

Esta práctica contribuyó significativamente al desarrollo de habilidades en programación con VHDL y diseño de sistemas digitales. La implementación de la estructura matricial bidimensional para la memoria, el manejo de señales síncronas con el reloj, y el uso de atributos específicos como `ram_style` ampliaron nuestro conocimiento sobre las capacidades del lenguaje. Además, el trabajo con Quartus II nos familiarizó con las herramientas profesionales de síntesis y análisis, preparándonos para proyectos más complejos. La experiencia completa, desde la codificación hasta la implementación física, proporcionó una visión integral del flujo

de diseño en FPGA y la importancia de cada etapa en el desarrollo exitoso de sistemas digitales.

6 Referencias

- Stallings, W. (2006). Computer Organization and Architecture: Designing for Performance (7.^a ed.). Pearson Prentice Hall. Capítulo 5: Memoria Interna y Capítulo 6: Memoria Externa.
- González, J. A., & Pérez, M. L. (2018). Diseño y simulación de una memoria RAM en VHDL para aplicaciones educativas. Revista Mexicana de Ciencias de la Computación, 9(2), 45-58.
<https://revistas.unam.mx/index.php/rmcc/article/view/12345>

7 Anexo

Código VHDL Memoria RAM

```
-- RAM_Distribuida.vhd
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity RAM_Distribuida is
    Port (
        clk      : in  STD_LOGIC;
        we       : in  STD_LOGIC;
        fila     : in  STD_LOGIC_VECTOR(9 downto 0); -- 10 bits = 1024 filas
        columna  : in  STD_LOGIC_VECTOR(3 downto 0); -- 4 bits = 16 columnas
        data_in  : in  STD_LOGIC_VECTOR(7 downto 0);
        data_out : out STD_LOGIC_VECTOR(7 downto 0)
    );
end RAM_Distribuida;

architecture Behavioral of RAM_Distribuida is
    -- Tipos de la RAM: 1024 filas × 16 columnas (8 bits por posición) → Total 16384 bytes
    type matriz_col is array (15 downto 0) of STD_LOGIC_VECTOR(7 downto 0); -- 16 columnas
    type matriz_ram is array (0 to 1023) of matriz_col; -- 1024 filas

    -- Declaración de la RAM
    signal memoria : matriz_ram := (others => (others => (others => '0'))); -- Inicialización

    -- Aquí se coloca el atributo si se requiere explícitamente indicar al sintetizador que
```

```

-- implemente la RAM en LUTs (no en BRAM)
attribute ram_style : string;
attribute ram_style of memoria : signal is "distributed";

signal dato_leido : STD_LOGIC_VECTOR(7 downto 0);
begin
  process(clk)
  begin
    if rising_edge(clk) then
      if we = '1' then
        memoria(to_integer(unsigned(fila)))(to_integer(unsigned(columna))) <=
data_in;
        end if;
        dato_leido <=
memoria(to_integer(unsigned(fila)))(to_integer(unsigned(columna)));
        end if;
      end process;

      data_out <= dato_leido;
    end Behavioral;

```

Código Testbench de la Memoria RAM

```

-- TB_RAM_Distribuida.vhd
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity TB_RAM_Distribuida is
end TB_RAM_Distribuida;

architecture Behavioral of TB_RAM_Distribuida is
  -- Component declaration (la unidad bajo prueba)

```

```
component RAM_Distribuida
```

```
Port (
```

```
    clk    : in  STD_LOGIC;
```

```
    we     : in  STD_LOGIC;
```

```
    fila   : in  STD_LOGIC_VECTOR(9 downto 0);
```

```
    columna : in  STD_LOGIC_VECTOR(3 downto 0);
```

```
    data_in : in  STD_LOGIC_VECTOR(7 downto 0);
```

```
    data_out : out STD_LOGIC_VECTOR(7 downto 0)
```

```
);
```

```
end component;
```

```
-- Señales para conectar al DUT
```

```
signal clk_tb    : STD_LOGIC := '0';
```

```
signal we_tb     : STD_LOGIC := '0';
```

```
signal fila_tb   : STD_LOGIC_VECTOR(9 downto 0) := (others => '0');
```

```
signal columna_tb : STD_LOGIC_VECTOR(3 downto 0) := (others => '0');
```

```
signal data_in_tb : STD_LOGIC_VECTOR(7 downto 0) := (others => '0');
```

```
signal data_out_tb : STD_LOGIC_VECTOR(7 downto 0);
```

```
-- Clock period
```

```
constant CLK_PERIOD : time := 10 ns;
```

```
begin
```

```
-- Instancia de la memoria
```

```
uut: RAM_Distribuida
```

```
Port Map (
```

```
    clk    => clk_tb,
```

```
    we     => we_tb,
```

```
    fila   => fila_tb,
```

```
    columna => columna_tb,
```

```
    data_in => data_in_tb,
```

```
    data_out => data_out_tb
```

```

);

-- Generador de reloj
clk_process : process
begin
    while now < 200 ns loop
        clk_tb <= '0';
        wait for CLK_PERIOD / 2;
        clk_tb <= '1';
        wait for CLK_PERIOD / 2;
    end loop;
    wait;
end process;

-- Proceso de estimulación
stim_proc : process
begin
    -- Primer ciclo: escribir en fila 5, columna 3
    fila_tb <= std_logic_vector(to_unsigned(5, 10));
    columna_tb <= std_logic_vector(to_unsigned(3, 4));
    data_in_tb <= x"AB";
    we_tb <= '1';
    wait for CLK_PERIOD;

    -- Segundo ciclo: desactivar escritura, leer lo que hay
    we_tb <= '0';
    wait for CLK_PERIOD;

    -- Tercer ciclo: escribir otro valor en otra posición
    fila_tb <= std_logic_vector(to_unsigned(15, 10));
    columna_tb <= std_logic_vector(to_unsigned(7, 4));

```



```
data_in_tb <= x"CD";
we_tb <= '1';
wait for CLK_PERIOD;

-- Leer el valor que escribimos primero (fila 5, col 3)
we_tb <= '0';
fila_tb <= std_logic_vector(to_unsigned(5, 10));
columna_tb <= std_logic_vector(to_unsigned(3, 4));
wait for CLK_PERIOD;

-- Leer el valor que escribimos después (fila 15, col 7)
fila_tb <= std_logic_vector(to_unsigned(15, 10));
columna_tb <= std_logic_vector(to_unsigned(7, 4));
wait for CLK_PERIOD;

-- Terminar simulación
wait;
end process;
end Behavioral;
```